ChatGPT

Title: OverKill Hill P³™ Custom GPT Configuration & Knowledge Integration Guide
Subject: Designing Effective GPTs with Structured Instructions and Knowledge Bases (RAG)
Author: OverKill Hill P³ Team
Creation Date: November 30, 2025

Website: overkillhill.com
Contact: contact@overkillhill.com

OverKill Hill Custom GPT Configuration & Knowledge Integration Guide

Table of Contents

- **1. Introduction**
- **2. Understanding GPTs and Custom GPTs**
- 2.1. What is a GPT (Generative Pre-Trained Transformer)?
- 2.2. What is a Custom GPT?
- **3. The Role of Instructions in GPT Configuration**
- 3.1. Writing Effective Instructions
- 3.2. Common Instruction Pitfalls to Avoid
- **4. The Role of Knowledge Files in GPTs**
- 4.1. How Knowledge Retrieval Works (RAG Integration)
- 4.2. Best Practices for Knowledge Base Content
- **5. Combining Instructions and Knowledge**
- 5.1. Interplay Between System Instructions and Knowledge Base
- 5.2. Ensuring the GPT Uses the Knowledge Base
- **6. GPT Configuration Quality Spectrum**
- 6.1. Characteristics of a “Poor” GPT Configuration
- 6.2. Characteristics of a “Good” GPT Configuration
- 6.3. Characteristics of a “Great” GPT Configuration
- **7. Best Practices Checklist for Custom GPT Design**
- **8. Frequently Asked Questions (FAQs) and Examples**
- **9. Conclusion and Future Outlook**
- **Bibliography**

1. Introduction

Developing a **Custom GPT** (Generative Pre-Trained Transformer) involves more than simply giving an AI a task. It requires a careful configuration of the AI’s **instructions** (its guiding rules and persona) and its **knowledge base** (any custom data or documents we attach for it to reference). This guide serves as a comprehensive knowledge artifact on how to build, refine, and optimize a custom GPT, with a focus on **Retrieval-Augmented Generation (RAG)** techniques. It is intended to be both a human-readable manual

for prompt engineers and AI designers, and a machine-ingestable knowledge file for AI retrieval systems. The content is structured into semantically coherent sections, includes question-and-answer examples, and provides canonical use-cases to illustrate key points. Citations to authoritative sources are included throughout (with a full bibliography) to ground the guidance in proven best practices.

In the sections that follow, we will:

- Define what GPTs are and explain the concept of custom GPTs.
- Discuss how **instructions** shape a GPT's behavior and how to author effective instruction sets.
- Explain how **knowledge files** work to give GPTs extended information and context (the basis of RAG), and how to organize such files for optimal retrieval.
- Explore the interaction between instructions and knowledge, including strategies to ensure the model uses its provided knowledge when answering.
- Compare examples of **poor, good, and great GPT configurations** to understand what differentiates a subpar setup from an excellent one.
- Provide a checklist of best practices and a FAQ section addressing common questions (with examples) for quick reference.

By the end of this guide, readers (and the GPT itself, if this document is used as its knowledge) should understand how to configure a GPT that is reliable, accurate, and tailored to a specific purpose, avoiding common pitfalls and leveraging the full power of instructions + knowledge integration. We begin with the fundamentals – understanding what GPTs are and how custom GPTs extend the capabilities of base models.

2. Understanding GPTs and Custom GPTs

2.1. What is a GPT (Generative Pre-Trained Transformer)?

A **GPT**, or *Generative Pre-Trained Transformer*, is a type of large language model (LLM) based on the transformer architecture [1]. In essence, GPTs are advanced AI systems trained on vast amounts of text data to predict and generate human-like text. OpenAI introduced the GPT series (GPT-1, GPT-2, GPT-3, GPT-4, etc.) between 2018 and 2023, demonstrating increasingly powerful language understanding and generation capabilities [2]. These models are called "**pre-trained**" because they undergo extensive training on general text corpora before they are fine-tuned or used for specific tasks, and "**transformer**" refers to the neural network architecture (from the seminal paper *"Attention Is All You Need"*, 2017) that enables efficient processing of language by attending to context [3].

Key characteristics of GPT models include:

- **Generative Capability:** GPTs can produce coherent and contextually relevant text outputs given an input prompt. They excel at tasks like answering questions, writing essays, summarizing documents, generating code, and more [4] [5]. The model predicts the next word in a sequence, allowing it to generate sentences and paragraphs that continue from the prompt in a fluent manner [6] [7].
- **Pre-trained Knowledge:** Because of extensive pre-training on diverse internet text, a GPT contains a broad range of general knowledge and linguistic patterns. GPT-4, for example, was trained on billions of words of text and can perform impressively on subjects from history to science. However, this training data has a cutoff (e.g., for GPT-4 it was mostly 2021 data), so the model may lack awareness of very recent events or niche documents unless provided via updates or external data.

- **Adaptability:** As *foundation models*, GPTs can be adapted to many tasks. With appropriate prompting or fine-tuning, the same model can switch from writing creative fiction to answering technical support queries. The prompt (including any system instructions, conversation history, and user query) guides its behavior for a given session.

Why are GPTs important? They represent a paradigm shift in AI, demonstrating that one large model can solve a wide array of language tasks without task-specific programming [3]. Companies and developers leverage GPTs (and derivative models from other providers like Anthropic's Claude or Google's PaLM/Bard) to build chatbots, virtual assistants, content generators, and other AI-powered applications. GPT-based chatbots like ChatGPT have brought this technology to mainstream use, showcasing how an AI can simulate conversational interactions and provide information or services in a natural language format.

Despite their power, GPT models also have limitations: they do not have **grounded, real-time knowledge** of facts beyond their training data unless augmented; they can produce *plausible but incorrect* statements (a phenomenon known as hallucination); and they follow the patterns of their training data, which includes biases and gaps. These limitations are exactly what **Custom GPTs with instructions and knowledge bases** aim to address – by constraining and informing the model with custom rules and up-to-date or domain-specific information. In the next subsection, we explain what Custom GPTs are and how they differ from the base models.

2.2. What is a Custom GPT?

A **Custom GPT** is essentially a tailored version of a GPT model (such as ChatGPT) configured for a specific purpose or domain. OpenAI's platform allows users to create custom GPTs by **combining extra instructions, custom knowledge, and selected capabilities** to specialize the AI's behavior [8]. In simpler terms, instead of using a general-purpose ChatGPT model "as-is" for every query, you can create your own GPT that has a persistent set of rules/instructions (like a persona or role), and optionally a set of uploaded reference documents (the knowledge base), along with tool integrations (like web browsing or code execution) if needed.

According to OpenAI's documentation, *"GPTs are custom versions of ChatGPT that users can tailor for specific tasks or topics by combining instructions, knowledge, and capabilities."* [8]. In practice, this means you can instruct the GPT to behave in a certain way or follow certain formats consistently, and supplement its built-in knowledge with specific data (e.g. your company's internal policies, a product manual, research papers, etc.). As a result, a custom GPT becomes an expert on a particular subject or function, effectively turning a general AI into a domain-specific assistant.

Why create a custom GPT? There are several motivations:

- **Consistency and Efficiency:** If you frequently use GPT-4 to perform a specific task (for example, act as a programming tutor or answer questions about a product), you would normally have to repeat the same context or instructions each time you start a new chat. By creating a custom GPT with those instructions preset, you save time and ensure consistency. *In essence, a custom GPT serves as a "prompt shortcut," remembering the context so you don't have to prompt from scratch every time* [9].
- **Domain Expertise:** The base GPT knows a little about a lot, but your custom GPT can be fed with detailed documents about your domain. This makes it far more knowledgeable (and accurate) in that niche. For instance, a custom GPT for medical support can have medical guidelines and drug

databases in its knowledge files, or a customer support GPT can include the company's FAQ and troubleshooting guides. The model will then use this *additional context* when formulating answers [10].

- **Controlled Behavior:** With carefully written instructions, you can control the tone, style, and even reasoning approach of the GPT. For example, you might instruct a custom GPT to always respond in a friendly tone and to never provide financial advice, or to always follow a specific step-by-step analytical method for problem-solving. This level of control is crucial for deploying AI in professional or sensitive contexts, ensuring the AI adheres to desired standards and policies.
- **Integration of Tools:** Custom GPTs also allow enabling certain **capabilities** or tools: e.g., web browsing for up-to-date information, code execution for data analysis, or custom APIs for specialized actions [11]. By selecting capabilities, the GPT can do more than just chat – it can fetch real-time info, generate images, run computations, etc., if those are enabled. This makes custom GPTs versatile agents tailored to certain workflows.

How is a Custom GPT constructed? When building one (via OpenAI's GPT Builder interface or similar tools), you typically configure several components: an **Instruction** set (the "system" message or persona/behavior guidelines), optionally some **Prompt Starters** (example user queries to demonstrate usage or provide opening context), the **Knowledge Base** (uploading files), and toggling on any **tools/capabilities** needed (like web access or plugins) [12] [13]. You also give it a name, description, and an avatar/icon to identify it in the interface [14]. Once published, you (and if shared, others) can chat with this GPT and it will maintain the configured persona and use the provided knowledge for answering.

Overall, a custom GPT can be thought of as *an AI specialist* you create: it knows the specific knowledge you give it and follows a set of rules or style you've designed. In the following sections, we will delve deeper into the two cornerstone elements of a custom GPT configuration – **Instructions** and **Knowledge Files** – and how to optimize each for best performance.

3. The Role of Instructions in GPT Configuration

Instructions (often called the **system prompt** or **instruction set**) are the written guidelines that define how the GPT should behave. Think of instructions as the *rulebook or persona* given to the AI. They can include the AI's role, its tone or style, what to emphasize or avoid, formatting requirements for outputs, and specific step-by-step processes to follow. Writing effective instructions is a form of **prompt engineering**, and it is crucial for steering the model towards reliable and desired outputs [15].

A well-crafted instruction set can dramatically improve a GPT's performance on a given task. Conversely, poorly written or insufficient instructions may result in the GPT misunderstanding its role, producing irrelevant or inconsistent answers, or ignoring important constraints. In this section, we discuss how to write effective instructions and highlight some common pitfalls to avoid.

3.1. Writing Effective Instructions

Clarity and Specificity: The instructions should clearly communicate the GPT's intended role and the scope of what it should do. Ambiguity is the enemy. For instance, simply instructing *"You are a helpful assistant"* is very general and leaves much to the model's discretion. If the goal is to have a GPT that provides, say, medical advice within safe limits, the instructions should explicitly state that role and boundaries (e.g., *"You are a virtual medical assistant providing general health information. You do not give a diagnosis or prescribe*

medication; for any urgent or specific concerns, you advise seeing a doctor," etc.). **The more precisely you define the task and persona, the less room there is for the model to go off-course.** Experts emphasize starting with a *clear use case*, noting that custom GPTs work best when *"laser-focused on a specific task or purpose"* [16]. Defining what the GPT is supposed to do, who will use it, and how it differs from a default general model will guide all other aspects of configuration [17]. In short, clarity up front leads to better-focused instructions and even reduces hallucinations [16].

Structured Formatting: Organizing the instruction text with headings, lists, and sections can significantly improve readability *for the model*. Even though the model doesn't "see" formatting in a visual sense, it does process the text tokens and benefits from well-structured prompts. OpenAI's guidelines suggest using Markdown formatting in instructions – such as headings (`#`, `##`), bullet points, and numbered lists – to delineate different parts of the instruction clearly [18]. For example, you might break your instructions into sections with headings like "Role", "Guidelines", "Steps", "Style", etc. Under each, use bullet points for individual rules. This segmentation helps the model attend to each rule distinctly. A heading might look like:

```
# Role
You are an AI financial advisor... (details of role)

# Guidelines
- Always respond in a reassuring and professional tone.
- Do **not** provide any speculative investment advice.
- ... (more rules)

# Format
1. Provide a brief summary, followed by bullet points of options.
2. End with a polite sign-off.
```

By structuring instructions into logical sections, you reduce the chance that the model skips or conflates instructions [19] [20]. Using **bold or italics** to highlight critical points (for example, **"ALWAYS double-check calculations before presenting results."**) can also help key instructions stand out [21] [22]. The OpenAI help article specifically encourages the use of Markdown headings and bullet lists in instructions to improve the model's adherence [23] [24].

Step-by-Step Procedures: If the task the GPT must perform involves multiple steps or a reasoning process, it's beneficial to spell that out explicitly in the instructions. For example, instruct the model in a numbered list to follow a certain workflow each time. *Trigger-action pairs* can be used: e.g., *"Trigger: User asks a question. Instruction: First, search the knowledge base for relevant info. Trigger: After gathering info. Instruction: Draft a concise answer with sources."* Breaking complex tasks into simpler sub-tasks improves accuracy [25]. One approach mentioned in guidance is to use "trigger/instruction pairs" separated by delimiters to ensure the model doesn't merge or skip steps [26]. This essentially simulates a flowchart in text, telling the model exactly what to do at each stage.

Positive phrasing and emphasis: It is generally recommended to phrase rules as positive directives ("do X") rather than negative ("don't do Y") where possible [27]. Models can sometimes misinterpret or over-correct when given negated instructions. Additionally, explicitly **emphasize critical instructions**. For example, you might include a reminder like: *"Take your time: Before finalizing an answer, double-check that you have used all*

relevant information from the knowledge files." Techniques such as telling the model to "take a deep breath" or "check your work" in the instructions have been noted to encourage thoroughness [21]. These kinds of meta-instructions nudge the model toward careful behavior rather than rushing to a response.

Few-Shot Examples in Instructions: If there are specific formats or decision criteria the GPT should use, providing a couple of example prompts and ideal answers in the instruction can be extremely helpful. This is called *few-shot prompting*. For instance, if you want the GPT to answer in a certain template, you can show:

- **Example User Query:** "How do I reset my password?"
- **Example GPT Answer:** "*(Step-by-step solution with a particular style)*".

Few-shot examples in the instruction act as demonstrations of the desired behavior or output. However, be careful to clearly separate these from the rest of instructions (using delimiters or a separate section) so the model recognizes them as examples, not something to literally include in its answer [28]. Also avoid overlapping content between instructions and knowledge: do not copy large chunks of source text into instructions; if it's reference info, put it in the knowledge file instead (more on this in Section 4). One **important rule** from experts: *"NEVER repeat the same content in both Instructions and Docs (knowledge files)"* [29] – duplicating information can confuse the model or waste context space.

Reinforcement of Model's Behavior: Sometimes adding a short **rationale** in instructions for why the model should follow them can indirectly improve compliance. For example: *"Provide a detailed rationale for each recommendation (this ensures transparency and builds user trust)."* While the model doesn't truly need a reason to obey, phrasing instructions with a purpose can align with how the model was trained (it often saw text explaining things). Additionally, instructing the model to reflect on the query or to think step-by-step (e.g., "always reason through the problem before answering") can trigger more complex reasoning pathways. This can be vital for complex tasks. In fact, after an OpenAI update introduced a new model routing system in 2025, many users found they had to be **much more explicit and demanding in instructions** to get the model to engage the more "intelligent" reasoning modes [30] [31]. For example, instead of a simple "You are an expert," an improved instruction might say: *"You are a world-class expert in X. For every prompt, you MUST perform a multi-step analysis: first, do …, second, do …, finally, present answers in N words."* – thereby *forcing* a deeper reasoning process [32]. This kind of specificity not only guides the model logically but can also signal the underlying system to allocate more computational effort, resulting in more nuanced responses.

Example – Instruction Excerpt: To illustrate, here is a condensed example of what a structured instruction set might look like for a hypothetical custom GPT designed to be a **Climate Policy Advisor**:

```
# Role
You are **ClimatePolicyGPT**, an AI expert on climate change policy and
economics. You assist policymakers by analyzing proposals and providing
insightful, fact-based answers.

# Knowledge Usage
- Always check the provided knowledge base documents (ClimateReports.pdf,
EmissionsData.pdf) for relevant facts and statistics.
- Prefer the knowledge base over your own assumptions for any specific data.
```

```
- If a question is outside the domain or not covered by the knowledge, politely
say you are unsure rather than guessing.

# Response Guidelines
1. **Tone:** Professional and neutral, as if addressing a governmental
committee.
2. **Format:** Start with a one-line summary, then provide detailed analysis in
paragraphs. Use bullet points for lists of options or recommendations.
3. **Citations:** When referencing data or studies from the knowledge files,
cite the source (e.g., "according to the 2023 Climate Report"). Do not reveal
file names, just refer to the document context.

# Step-by-Step Approach
- *Step 1:* Carefully read the user's question and identify the key issue (e.g.,
emissions target, economic impact, etc.).
- *Step 2:* Search the knowledge base for relevant information (e.g., specific
data points, case studies).
- *Step 3:* Formulate an answer that addresses the question directly, backed by
facts from the knowledge.
- *Step 4:* Conclude with a concise recommendation or summary.

# Avoid
- Don't provide financial investment advice.
- Don't take political sides; remain factual and unbiased.
- If asked for figures and none are in knowledge, say that the data is not
available rather than guessing.
```

This example demonstrates various elements: it sets a role and persona, it instructs on knowledge usage (naming specific files and how to use them), it outlines how to format and tone the answers, provides a step-by-step method, and explicitly lists what *not* to do. A real instruction set might be even more detailed, but this gives a flavor of a structured prompt that a **great configuration** would have. As the example shows, good instructions often anticipate potential pitfalls (e.g., being asked something outside scope) and pre-emptively guide the model on how to respond.

Testing and Iterating Instructions: Writing instructions is not a one-shot task. It's advisable to test your custom GPT with a variety of user questions and see if it behaves as expected. If not, refine the instructions. OpenAI provides a "GPT Builder" tool that can even highlight potential issues or provide suggestions in your draft instructions [33]. Questions to ask during testing: *"Is this GPT behaving how I expect? Is it being helpful, or just verbose?"* [34]. If certain user queries lead to undesired answers, consider adjusting the instructions to cover those cases. For example, if the GPT gave too brief an answer, you might add "Always provide at least three detailed paragraphs in your responses." If it gave an opinion where it shouldn't, add "Do not offer personal opinions, stick to analysis." This iterative tightening of instructions is key to moving from a **good** configuration to a **great** one.

In summary, effective instructions are **clear, structured, and explicit**. They act as the internal compass for the GPT. With well-written instructions, you set the stage for success – but the instructions alone are not enough if the GPT lacks information. That's where the **knowledge files** come in, which we cover next.

3.2. Common Instruction Pitfalls to Avoid

Even with the best intentions, it's easy to accidentally write instructions that are suboptimal or even counterproductive. Here we list some common pitfalls in GPT instructions and how to avoid them:

- **Vague or Overly General Instructions:** As mentioned, instructions like "You are a helpful assistant" or "Answer questions as best as you can" are too general. They don't provide guidance on *how* to be helpful or what sources to use. Vague instructions leave the model to fill gaps with its own defaults, which might not align with your needs. **Solution:** Be specific about the role and task. Instead of "be helpful," specify what being helpful means in context (e.g., "provide step-by-step solutions for math problems and check the work for errors"). Instead of "as best as you can," detail the criteria for a good answer (e.g., "comprehensive, uses simple language, and includes an example if applicable").

- **Conflicting or Redundant Rules:** If instructions contain statements that contradict each other, the model may become confused about which to follow. For example, one line says "Be concise" and another says "Provide detailed explanations." The model might flip-flop or default to one. **Solution:** Ensure your instructions are internally consistent. If you need a balance (like detailed yet concise), clarify what that means (e.g., "Provide detail but avoid irrelevant digression – aim for 2-3 focused paragraphs."). Also remove any duplicate instructions to save space and avoid confusion.

- **Excessively Long or Complex Instructions:** While it's important to be thorough, a huge wall of text with too many rules can be counterproductive. The model has a context window limit and if instructions are extremely long, it might truncate or pay less attention to the later parts, especially if not structured. Complex conditional instructions ("If the user asks X do Y unless Z, but if A then do B…") can also cause issues if not clear. **Solution:** Keep instructions as succinct as possible while covering the necessary points. Use bullet points and short sentences. Break complex logic into separate rules or steps. It might be better to have 10 clear bullet points than one convoluted paragraph trying to cover all edge cases.

- **Negative Instructions Emphasis:** Saying "Do not do X" repeatedly could sometimes perversely lead the model to mention or fixate on X (this is akin to the psychological "avoid thinking of a pink elephant" problem). **Solution:** Whenever possible, frame things positively. Instead of "Don't write in first person," you could say "Always write in third person." Instead of "Do not use informal language," say "Use formal, professional language." If negative instructions are needed (some definitely are, like to forbid certain content), you can include them but try not to list a dozen "don'ts" without any positive guidance or the model might get tunnel-vision on what it shouldn't do and struggle with what it *should* do.

- **Unrealistic Absolute Rules:** Instructions like "Always be 100% correct" or "Answer any question perfectly" are not practical – the model will do its best, but if something is impossible it might lead to forced or nonsense output. Telling the model "never apologize" or "never refuse a request" can also backfire when a refusal *is necessary* (e.g. if user asks disallowed content). **Solution:** Keep rules realistic and if there are circumstances where they don't apply, acknowledge them. For instance, "Aim to be as accurate as possible; if you are unsure or the information is not available, admit that rather than guessing." That sets a practical expectation.

- **Ignoring the "Enable" Settings:** Some behaviors like web searching or code execution require enabling those capabilities on the platform side. No matter how much you instruct the GPT to browse the web, if the capability isn't turned on, it won't do it. Conversely, if you enable a tool, ensure your instructions mention when/how to use it. **Solution:** Align instructions with enabled tools. For example, if web browsing is enabled, you might instruct, "If a query requires up-to-date information not found in knowledge, use the Browse tool to search online." If code interpreter is enabled, instruct when to use it (e.g., for math calculations or data file analysis). Also, remember that by default certain features like browsing may be off; the user (or GPT builder) must toggle them on. The instructions can remind *to* use them, but cannot activate them on their own.

- **Using the "Create" Tab Suggestions Unedited:** OpenAI's interface has a "Create" mode where ChatGPT will interactively help build the GPT by asking questions. While this is a convenient starting point, it may not yield the most optimized instructions. Users have noted that the Create tab may not allow full control over formatting or logic [35]. **Solution:** Treat the AI-generated instruction draft as a first draft. Review and edit it in the Configure tab manually. Ensure formatting, consistency, and completeness. The AI might miss context or produce generic text – customize it with the specifics of your use-case.

By being mindful of these pitfalls, you can refine your instruction set to avoid confusion and misbehavior. Good instructions, as a rule of thumb, read like a clear manual or checklist for the model. They might even be understandable to another human as the "job description" for the GPT. In the next section, we'll shift perspective to the other side of the equation: how to manage and format the **knowledge files** that supply your GPT with factual content.

4. The Role of Knowledge Files in GPTs

One of the most powerful features of a custom GPT is the ability to provide it with **Knowledge Files** – essentially, custom data that the GPT can draw upon when answering questions. This is the essence of **Retrieval-Augmented Generation (RAG)**: the model retrieves relevant information from an external repository (the knowledge base) and uses it to generate more accurate or context-specific responses. In OpenAI's implementation, you can upload up to 20 files to your GPT, each file up to 512 MB or around 2 million tokens in size [36]. These files can be plain text, PDFs, or other formats (images can be uploaded but only extracted text is used, since the model currently processes text).

Importantly, the GPT does not ingest these files into its permanent memory; instead, it indexes them so that it can search for relevant content **on the fly** when a user prompt comes in [37]. In this section, we discuss how knowledge retrieval works under the hood, and how to prepare knowledge files for optimal results.

4.1. How Knowledge Retrieval Works (RAG Integration)

When you attach knowledge files to a GPT, the system performs a preprocessing step: it breaks each document's text into smaller pieces (often called **chunks**) and creates an **embedding** for each chunk [37]. An embedding is a vector (a list of numbers) that semantically represents the content of that chunk. This allows the system to compare the semantic similarity between a user's question and parts of the documents quickly. The collection of embeddings acts like an index or knowledge vector database behind the scenes.

Here's what happens at query time, in simplified terms:

1. **User prompt arrives.** The user asks a question or makes a request. For example: "What are the main benefits of implementing solar energy according to the 2023 climate report?"
2. **Semantic search on knowledge base.** The GPT system will take the user's query and generate an embedding for it, then compare that embedding to all the chunk embeddings from the knowledge files to find the most relevant pieces of text. This is known as a **semantic search** or similarity search [38]. The idea is to retrieve the chunks of your documents that closely relate to the question asked.
3. **Retrieval of relevant text chunks or documents.** Depending on the nature of the query, the system may retrieve one or several chunks of text and include them as context for the model, or it might determine that an entire document (or a large excerpt of it) is needed. According to OpenAI's documentation, for straightforward Q&A style queries where a specific answer is located in the sources, the system will return the *relevant text chunks* [39]. This ensures the model has the specific snippet (for instance, a definition or a fact) to base its answer on. For tasks like summarization or translation of an uploaded document, the system can employ a *document review* mode, returning an entire short document or the relevant section of a big document to the prompt [40]. This way, if the user says "Summarize the attached policy document," the GPT will have the full text (or at least the major portion) of that document in its working context to summarize.
4. **Answer generation using retrieved context.** The GPT model, now with the user's question plus the additional context from the knowledge file, will generate its answer. Essentially, the relevant chunk(s) from your files are appended to the prompt (invisibly to the user) in a special format. The model sees something like: "[Document Excerpt]: … (text from file) … [User question]: …". It then uses both the excerpt and its own internal knowledge to compose the response. Ideally, it will rely primarily on the provided excerpt for factual details, since that's likely the most relevant and up-to-date source. This hybrid of recall and generation is what makes the answer both accurate and coherent.
5. **Citations or references (if instructed).** By default, the model will use the info but not reveal the source file names [41]. However, you can instruct it to cite the source (e.g., "according to the Company Handbook") if you want transparency. It won't automatically produce citations unless told to, because the system is designed to avoid exposing file names or raw content unless necessary [42]. With proper instructions, it can mention the document title or other identifying info instead of a cryptic file name.

It's worth noting that the GPT does **not simply copy-paste** entire sections of your knowledge base into every answer. It picks what seems relevant. One user describes it as the model *"integrates [the knowledge files'] content as background knowledge. It does not copy verbatim but instead applies the information contextually when generating responses."* [43]. In other words, the knowledge is there to be woven into the answer as needed, rather than dumped wholesale. If you gave the GPT a methodology document, it will use the methods from that document in its reasoning or suggestions *rather than quoting it directly* [43] – unless of course the question is "quote the exact text".

When does the GPT choose knowledge vs other methods? The system's logic for whether to inject knowledge, use web search, or just rely on its own training can be complex, but as of the latest design, if knowledge files are attached, the GPT will try to use them for relevant queries. If the knowledge files don't contain information relevant to the question (e.g., user asks something completely unrelated to any provided document), the GPT might then fall back to its built-in knowledge or use a tool like web search (if enabled) to find the answer. But ideally, for the scope of the custom GPT, your knowledge files cover the typical queries expected. The OpenAI help suggests that you *"encourage the GPT to rely on Knowledge first before searching the internet."* [41] via instructions. This is a good practice so that, for example, your company

GPT doesn't ignore the very policy file you attached and go look up some generic info on the web. We will discuss this interplay in Section 5.

Additionally, because knowledge retrieval consumes tokens and time, there may be some internal thresholding. If a user's query is very simple and clearly not related to any uploaded content, the system might not waste effort retrieving documents. However, if it's a question that even might have an answer in the files, it will attempt to fetch it. Users have observed that sometimes a custom GPT may *not* use the knowledge files unless the query is specific enough or unless explicitly prompted to. That's why guiding it in the instructions (like "When answering, check the knowledge files for relevant info before responding") can be helpful.

The benefits of this retrieval approach are significant: it effectively extends the GPT's knowledge beyond its training cutoff or scope, without requiring fine-tuning. The model remains the same, but it's augmented with a dynamic memory (the knowledge base). This approach also provides a measure of correctness and updatability – if the domain knowledge changes, you can update the files without retraining the model; the next query will use the new information.

However, the success of retrieval-augmented answers largely depends on **how well the knowledge is structured and provided**. If the knowledge files are disorganized, too large and unwieldy, or irrelevant, the GPT might retrieve the wrong information or too much information, leading to poor answers or slow responses. This leads us to best practices in preparing knowledge files.

4.2. Best Practices for Knowledge Base Content

Not all knowledge bases are created equal. Simply uploading a 500-page PDF of raw text might technically give the GPT a lot of information, but it may struggle to find the needle in the haystack quickly. On the other hand, a carefully curated and segmented knowledge base can make the GPT extremely efficient and accurate in fetching relevant info. Here are some best practices when assembling knowledge files for a custom GPT, informed by both official guidance and community experiences:

- **Structure and Chunking:** As noted earlier, the system will chunk your documents for embedding. You can make its job easier by structuring the documents with clear sections, headings, and paragraphs. Large continuous blocks of text are harder for the model to navigate. One experienced user reported, *"I've worked with over 25,000 words across multiple documents without issues, as long as the content is well-organized. Performance tends to slow down when the information is disorganized, redundant or lacks clear segmentation."* [44]. The takeaway is that **clarity and segmentation trump brevity**. A 10,000-word well-structured file can outperform a poorly-structured 2,000-word file in terms of retrieval efficiency [45]. Use an outline, insert logical headings, break long paragraphs into shorter ones. If the content is a list of facts, consider bullet points or tables for clarity. If it's a narrative, ensure each major point starts a new paragraph or section. Remember, each heading or paragraph likely becomes a chunk or helps define chunk boundaries.

- **Relevance and Coverage:** The knowledge files should cover the scope of questions you expect the GPT to handle. It's okay to have multiple files for different subtopics; in fact, it can be beneficial to break knowledge into modular files (e.g., "Product Specs", "Company Policies", "Troubleshooting Guide", etc.) rather than one monolithic document [46]. However, avoid throwing in extraneous information that isn't needed. Irrelevant content could be erroneously retrieved or might use up

context space. Aim for each file to be topically coherent and clearly titled (the file name or content title can be referenced in instructions and helps you keep track).

- **Avoid Redundancy:** Don't include the same information in multiple files or repeated within a file unless necessary. Redundant info not only wastes space but can confuse the retrieval (the system might find multiple very similar chunks and include them all, which doesn't add new value). If you have overlap between documents, consider consolidating that into one place. This also ties to the earlier point from Adam Mico's guidance: if something is in the knowledge file, you generally don't need to also paste it into the instruction prompt (no duplication) [29].

- **Formatting of Files:** Preferred formats for the content are **plain text or simple PDF**. The OpenAI parser that extracts text works best on documents with simple layouts [47]. A single-column text PDF, Markdown, or text file is ideal. Avoid complex formatting like multi-column newsletters, heavy use of tables or weird fonts, or scanned images of text (which might not OCR well). If you have a multi-column PDF (like academic papers or slides), consider converting or reformatting it to a single column text for better ingestion. Also, be mindful that things like footnotes, headers/footers on pages, and irrelevant metadata in documents could become noise in the text. Where possible, clean the text before uploading (e.g., remove repeated page headers, table of contents pages that aren't needed, etc., unless you specifically want them searchable).

- **Use Markdown in Knowledge (if possible):** If you have the ability to provide knowledge in Markdown (as this document is written), do so. Markdown uses plain text with simple conventions that the parser can easily interpret. It preserves headings (#, ##), lists, bold/italic etc., which can become cues for the model. A community member noted, *"Markdown is great for structuring content since GPT processes it cleanly... GPT might not interpret complex PDF formatting as effectively as plain, well-structured markdown."* [45]. So, if you have a document, consider converting it to a Markdown (.md) file with clear heading structure and then upload that, instead of a PDF with fancy styling. The content remains the same, but the structure is explicit in text form.

- **File Size and Chunk Size Considerations:** Each file can be very large (hundreds of pages) but bigger isn't always better. If one file is extremely large, the system will break it into many chunks and searching through them could be slower. Moreover, if a file contains lots of different topics, it might retrieve chunks from that file even if another more targeted file would have been better. A strategy is to break large files into smaller ones by topic. For example, instead of one 200-page "Manual.pdf", split it into "Manual_Part1_Overview.txt", "Manual_Part2_Features.txt", etc., or even by chapters. This way, if a question pertains to a specific part, the system might only retrieve from the relevant file. That said, avoid too many tiny files with only a sentence or two; a balance is needed. Up to 20 files are allowed [36], which is usually plenty. If you have a vast amount of data, consider what a user is likely to ask and prioritize including information that answers those likely questions.

- **Content Style – Lists vs Paragraphs:** Depending on information type, consider the format. For factual data or steps, bullet lists or numbered lists make it easy for the model to pull a specific point. For explanatory content, paragraphs are fine. A mix is often best: use paragraphs for detailed explanations and lists for summarizing key points [48]. For instance, a knowledge file on "Benefits of Solar Energy" might start with a paragraph explaining general context, and then have a list: "- Reduces greenhouse gas emissions. - Lowers electricity costs in long term. - Scalable from small to

large installations." If the user asks "What are the benefits of solar energy?", the semantic search might latch onto "benefits" and pull that list directly.

- **Keep Knowledge Up-to-Date:** If your GPT is being used in a context where information changes (e.g., a knowledge base for software features that get updated, or laws that change), make sure to update the files accordingly. The GPT won't magically know of changes unless the files are replaced with new content. Currently, the only way to update is manually via the UI (uploading new files) [49], so plan for periodic maintenance. It's often helpful to include a version number or date in the document or file name (e.g., "Policy_Handbook_Oct2025.md") so you know which version is loaded.

- **Test the Knowledge Retrieval:** Just as we test instructions, test the knowledge integration. Ask the GPT a question that should be answered from the knowledge file and see if it indeed uses it. If it hallucinates or ignores the file, check if: (a) the question wording matches how the info is phrased in the file (if not, consider adding synonyms or tweaking phrasing in the file or ask the question differently in instructions as a sample), (b) ensure the instruction does not inadvertently tell the model to ignore knowledge (it shouldn't, but just in case any conflicting guidance is there), and (c) the knowledge chunk might be too large to fit in context if it's pulling a whole doc – in such cases summarization mode might cut details. In community discussions, one user pointed out that *readability and structure are key to help GPT "extract relevant information efficiently."* [48] If the model struggles, consider subdividing the content further or rewriting particularly important sections in a more question-and-answer form inside the doc (like an FAQ format) to aid retrieval.

- **Encourage Proper Use of Knowledge via Instructions:** Although this bleeds into interplay (section 5), it is worth noting here: in the knowledge files themselves you typically just put reference content, not instructions. The instructions (system prompt) is where you say "use the knowledge first, cite it," etc. The knowledge file should not contain orders to the model; it should be pure info. However, you can include things like an "About this document" note if helpful (e.g., "This handbook covers company policies in 2025"). But leave the procedural guidance to instructions. In the next section, we will see how to tie these together.

In summary, a well-prepared knowledge base is **clear, organized, relevant, and as concise as it can be while still covering the necessary information**. When done right, your custom GPT will appear to have an almost expert-level understanding of your domain, because it can seamlessly pull in facts and details from the uploaded files. An added benefit is that it helps reduce hallucination – the model doesn't need to make up an answer if the answer is readily available in the documentation you've given it.

Next, we will discuss how **instructions and knowledge interact** and how to ensure your GPT effectively utilizes the knowledge base within the rules you've set.

5. Combining Instructions and Knowledge

Thus far, we've looked at instructions (the rules/persona for the GPT) and knowledge files (the reference information) mostly in isolation. In practice, a **great custom GPT configuration comes from a harmonious interplay between the two**. The instructions and the knowledge base should complement each other: instructions set the strategy and expectations, while knowledge provides the substance for factual or domain-specific queries. This section covers how to weave them together, ensuring the GPT actually uses the knowledge when it should, and understanding how one influences the other.

5.1. Interplay Between System Instructions and Knowledge Base

The **system instructions** effectively act as the manager or the reasoning guide for the GPT, whereas the **knowledge base** is like the library of facts at its disposal. A well-designed custom GPT will have instructions that explicitly tell the model *when and how to use that library*. This is crucial because, as noted, the model might not always automatically use the knowledge, especially if the query is phrased vaguely or it's borderline whether the knowledge is needed.

For example, consider a scenario: You have a custom GPT for IT support with a knowledge file containing troubleshooting steps for various software. If a user asks, "How do I reset my email password?", the answer likely lives in the knowledge file (assuming the file has a section on password resets). But if your instructions do not mention the knowledge at all, the model might just try to answer from its general training ("Usually, you go to settings and click 'Forgot Password'..."), which could be fine but might miss company-specific nuances or links in the knowledge file. By contrast, if your instructions say, "When a question relates to procedures in our IT policies, check the IT_Knowledge document for the official steps," the model is nudged to use the knowledge.

OpenAI's guidance directly states: *"Using Instructions in the GPT editor, you can encourage the GPT to rely on Knowledge first before searching the internet."* [41] . So a best practice instruction might be: "Always attempt to answer using the provided Knowledge documents before resorting to any external sources or your built-in knowledge. The knowledge files are up-to-date and should be considered the primary source of truth for their respective topics." This makes it clear to the model that Knowledge > internal guesswork for our purposes. If web browsing is enabled as well, you might add: "Use web search only if the answer cannot be found in the knowledge files."

Another interplay aspect: instructions can define **how to present information from knowledge files**. By default, the model might integrate the info without attribution. If you want it to **cite sources**, you must say so in the instructions. The OpenAI help notes that *"by default, GPTs will avoid revealing the names of uploaded files"* [42] , which means if your knowledge file is named "Policy2025.pdf", the model won't mention "Policy2025.pdf" in the answer unless instructed (to protect potentially private file names). If you want it to cite, you could instruct: "When providing factual answers from the knowledge, cite the document title or a reference so the user knows the source (e.g., 'according to the 2025 Policy Manual...'). Do not cite file names, but rather use a descriptive name for the source." This way, the model has permission to reveal something about the source. It might then say, *"According to the Employee Handbook, vacation accrual resets every calendar year,"* which is useful to the user to know the answer is from an official source. Always ensure to phrase it such that it's not exposing any confidential filename unless that's acceptable (perhaps renaming files to friendly titles can help, like having the file content itself start with "Employee Handbook (2025)" so the model can use that text as reference).

Consistency between Instructions and Knowledge: Make sure your instructions do not contradict what's in the knowledge files. For instance, if instructions say "Always give 3 options to the user's query," but the knowledge file has a section that implies only 1 solution exists for something, the model might be torn. Generally, the model will follow instructions on format and style, and use knowledge for content. But if instructions contained a factual statement that conflicts with knowledge, that's a problem. Ideally, put factual content in knowledge, not in instructions, to avoid such conflict. Instructions should be about *how* to use knowledge, not the knowledge itself (with few exceptions like maybe a critical piece of info you want front and center, but even that could be seen as a kind of knowledge).

Persona and Tone vs. Factual Data: Instructions often set a persona or tone (e.g., "You are polite, use technical language appropriately, etc."). The knowledge files contain raw info which might be dry or technical. It's the instructions' job to tell the model how to incorporate that info in the chosen tone. For example, your instructions might say "Explain any technical terms when giving answers to non-technical users." The knowledge file might have a sentence "The system uses a Zero Trust security model with SSO integration." If a user asks a related question, the model, guided by instructions, might answer: "Our system uses a **Zero Trust** security model (a framework where every access request is verified) along with **Single Sign-On (SSO)** integration to improve security [48]." Notice how it can pull the jargon from knowledge but then explain it, fulfilling the persona directive of being user-friendly.

Handling Missing Knowledge: Instructions should cover cases when the knowledge base doesn't have the answer. For instance, "If the answer is not found in the knowledge or it's a general question beyond our documents, you may use your general knowledge or politely say you don't have that information." This prevents the GPT from awkwardly trying to fabricate an answer that it isn't confident about. It's better for it to either use its own training data (with caution) or just say "I'm sorry, I don't have information on that" if that fits the scenario (depending on how you want it to handle unknowns). If web browsing is enabled as a backup, instructions can also mention: "If knowledge files don't contain the answer and the question is factual, use the Browse tool to find information."

Tool Use Instructions: If you have both knowledge and other tools, be explicit in how to prioritize. For example: "First check knowledge. If needed info isn't there and the question is about current events, use the Browse web tool. If calculations or file analysis is needed, use the Code Interpreter tool. Always explain your results clearly to the user." This layered instruction ensures the model doesn't, say, immediately go to web for something that was sitting in the knowledge file.

Interestingly, OpenAI's instruction writing tips suggest to *"Provide explicit instructions to use tools such as Browse, Knowledge, and Custom Actions throughout the instructions."* [50]. That means you might literally have a section in instructions like:

```
# Tools
- **Knowledge:** Use the Knowledge files for any factual questions about our
products or policies. (Files: ProductGuide, FAQ)
- **Browse:** If a question requires information not in Knowledge (like recent
news or external info), use web browsing to find the answer.
- **Calculator:** If a question involves a complex calculation or data analysis,
use the Code Interpreter to compute it, then explain the result.
```

This clarity helps the model know when to use what. If it wasn't spelled out, the model might guess or do things in a suboptimal order.

Effect of Instructions on Retrieved Chunks: The content of instructions can also slightly influence what gets retrieved. The retrieval is primarily based on user query vs chunk similarity, but if your instructions mention certain key terms a lot (especially in a way that ties to content), it might affect the model's bias on what is relevant. Generally, the system is designed such that instructions are separate from the query for the retrieval step, but after retrieval when the model is answering, it sees both. For example, if instructions emphasize "prefer knowledge for factual answers," that's more a behavioral guide post-retrieval. But if

instructions accidentally contained text similar to something in a knowledge file, it could—at worst—cause the model to think that content was already in context (this is a rare edge case and usually not an issue if your instructions don't contain large overlap with knowledge content).

5.2. Ensuring the GPT Uses the Knowledge Base

One common complaint early users of custom GPTs have is: *"My GPT isn't using the knowledge documents I uploaded!"* There are a few possible reasons and solutions for this, which we'll outline:

- **Reason 1: The question doesn't obviously match the knowledge content.** If the user's query is phrased in a way that doesn't overlap with how the info is written in the files, the semantic search might not find it. For instance, knowledge file says "Annual leave accrues on a prorated basis for part-time employees," and user asks "Do part-time workers earn vacation time gradually?" If the wording "vacation time" vs "annual leave" or "gradually" vs "prorated" don't semantically match enough, the system might not retrieve that chunk. **Solution:** You can address this by enriching your knowledge content with synonyms or alternative phrasings (maybe have an FAQ in the document that uses casual terms: "Q: Do part-time workers get vacation? A: Yes, part-time employees accrue vacation (annual leave) on a prorated basis…"). Also, instructions could detect or reformulate user queries—but that's advanced. At least ensure key terminology in your domain is covered in knowledge. You could even have a glossary file mapping synonyms, which could help retrieval if query uses one term and file uses another.

- **Reason 2: The GPT *thinks* it knows the answer without knowledge.** GPTs have a lot of pre-trained knowledge. If a user question triggers the model's general knowledge strongly, it might answer from that instead of taking the extra step to fetch from the custom data. For example, ask a general question like "What is the capital of France?" – the model "knows" this (Paris) and might answer immediately without any knowledge file (assuming you had a geography file or something). This is usually fine for common knowledge, but if your knowledge had a twist (say a file says "For the purpose of our fictional scenario, treat the capital of France as Lyon."), the model might ignore that unless instructed. **Solution:** The instructions should clearly set the expectation that for domain-specific or relevant questions, the knowledge is primary. Also phrase knowledge in a way that if a conflicting known fact exists, clarify the context (e.g., "**Note:** In reality the capital is Paris, but in this scenario, we assume it is Lyon."). If it's something factual like an internal number or policy that the model might also have an idea of, you must override it in instructions: e.g., "Although the model has general knowledge, for our purposes use the data in the files even if it conflicts with common knowledge."

- **Reason 3: The model wasn't explicitly told to use the knowledge.** Models often do use provided info if clearly relevant, but they might not always make the connection. If your instructions never mention the existence of knowledge files, the model might not "realize" it has extra info to draw from. **Solution:** As stressed, explicitly instruct to use the knowledge. For example: "You have access to the following documents in the knowledge base which contain authoritative information. Always check them when relevant to the user's question." Many have found that a simple line like this in instructions boosts usage of knowledge significantly. It's akin to reminding the AI: "hey, use the library at your disposal." Without it, it might follow some default heuristic which could be less aggressive in retrieval.

- **Reason 4: Knowledge retrieval returned results but model ignored or misused them.** Sometimes the system might fetch the chunks, but the model doesn't incorporate them well. This could happen if the answer formulation isn't straightforward, or if the retrieved text is long or complicated. **Solution:** You can encourage how to use retrieved info. For instance: "If information from knowledge is found, integrate it into the answer by paraphrasing or summarizing it – do not just copy large text verbatim. Ensure the answer directly addresses the user's query using those details." Also ensure instructions on format don't conflict; if you force a certain structure, make sure it still leaves room to include the knowledge details. Another tactic: Provide an example in instructions of a QA where the knowledge is used. E.g., "**Example**: User asks X. (In the knowledge file, Y is the answer). **Answer**: (Model gives answer referencing Y)". This demonstrates the expected behavior explicitly.

- **Reason 5: Too much knowledge or poorly optimized knowledge.** If the knowledge base is huge and not well-segmented, the retrieval might pull something irrelevant or partial, leading the model to maybe ignore it or get confused. Also, if many chunks are equally semi-relevant, the model might be unsure what to use. **Solution:** Fine-tune the knowledge content as per section 4.2. If an irrelevant chunk was being pulled often, consider why (maybe a keyword appears too broadly). If multiple similar answers are in different files, perhaps consolidate or differentiate them with context so the model picks the right one. In some cases, fewer but more precise documents yield better results.

- **Reason 6: Limitations or Bugs in the system:** It's possible (especially around 2024-2025 when custom GPT features are evolving) that sometimes the knowledge feature has hiccups – e.g., not all file text was indexed, or certain formats got parsed incorrectly. If something really should have been found and never is, try re-uploading in a simpler format. Or break the content differently.

To illustrate ensuring knowledge use, let's say our custom GPT is for a company HR assistant. A user asks: "How many vacation days do new employees get per year?" The knowledge file "HR_Policy.md" has the line: "Employees with less than 1 year of service receive 10 vacation days annually." If the GPT answers from knowledge, it should say: "New employees receive 10 vacation days per year, according to our HR policy." If it didn't use knowledge, it might say "Usually, companies give around 10-15 days." That's generic and not company-specific. We ensure it uses the file by having an instruction: "For any questions about company policy (e.g., benefits, holidays, etc.), refer to the HR_Policy document and use its exact figures." With that, the model is guided to check and thus will likely pull that exact number 10 from the file. Additionally, it might cite it as "according to the HR policy" if we asked it to cite.

Finally, **user prompt design** can also ensure knowledge usage. While this guide is more about the GPT's internal design, educating users or providing prompt starters can help. For example, a prompt starter could be: "Ask me anything about our company policies (e.g., 'According to the handbook, what is our dress code?')." That prompt itself hints the GPT to use the "handbook". But generally, if we do our job in instructions, the user shouldn't have to phrase things specially.

In summary, by explicitly instructing the model to leverage the knowledge, structuring both the knowledge and instructions to be complementary, and covering edge cases when knowledge might be missing or conflicting, we can maximize the synergy between the two. A fully optimized custom GPT will smoothly use its knowledge base as an extension of its brain, guided by the rules and style we've imposed via instructions. Next, we will examine the outcomes of putting all these pieces together by comparing poor, good, and great configurations, which will further solidify these concepts.

6. GPT Configuration Quality Spectrum

Not all GPT setups are equal. The difference between a hastily thrown-together custom GPT and a meticulously crafted one is like night and day in terms of performance, user satisfaction, and reliability. In this section, we characterize three broad categories of GPT configurations – **"Poor", "Good", and "Great"** – and highlight the characteristics of each. By understanding these, you can self-evaluate and iterate on your GPT design to push it from poor or mediocre toward great.

It's worth noting these categories are somewhat informal; there isn't an official grading, but they serve as a conceptual framework:

6.1. Characteristics of a "Poor" GPT Configuration

A "poor" GPT configuration is one that fails to meet the needs of its intended use-case in one or more significant ways. Such GPTs often produce disappointing results, and users may find them frustrating or "dumb." Common characteristics include:

- **Minimal or No Custom Instructions:** The GPT has either the default instructions (e.g., just "You are ChatGPT, a large language model…") or extremely brief instructions that do not clarify the role or guidelines. For example, only instructing *"You are an assistant that answers questions about our products."* This doesn't cover style, depth, or any specifics, so the model essentially behaves like a generic GPT with a slight hint of context. As a result, it may ignore important nuances or behave inconsistently.

- **No Emphasis on Knowledge Usage:** In a poor configuration, even if knowledge files are attached, the instructions might not mention them at all. The model might not use the knowledge unless the user explicitly asks something that directly matches it. Many answers could be generic, potentially incorrect, or lacking detail that was actually available in the files. For example, the knowledge file might have a detailed spec sheet, but the GPT still gives a generic answer like "Our product has various features to help you," because it didn't consult the data.

- **Unstructured Instructions or Content:** If the instructions are written as one long paragraph with multiple ideas jumbled together, the model might miss or ignore parts. For instance, "Answer questions about health. Be an expert. Don't be too verbose but also give details. And ensure to be friendly. Also, if the user asks for advice give it." – This is messy, and the model might latch onto one aspect (like being friendly) and forget "give details" or vice versa. Similarly, if knowledge files are just raw dumps (like an entire email thread copy-pasted, or a log of data without explanation), the model's retrieval might pull irrelevant lines.

- **Contradictory or Confusing Guidance:** A poor configuration might inadvertently contain conflicting instructions. For example, one part says "Elaborate on answers" and another says "Keep answers brief." Or the instruction says "always cite sources" but then the knowledge files' content isn't set up to cite (and instructions didn't clarify how), so the model might be torn and either not cite or cite in a weird way (maybe citing its own training data incorrectly). This inconsistency leads to erratic outputs – sometimes verbose, sometimes not; sometimes citing, sometimes not.

- **Lack of Focus (Scope Creep):** The GPT's purpose isn't clearly defined, so it tries to do too many things or doesn't excel at the main thing. For instance, a custom GPT meant to be a "Programming Helper" that also has random knowledge files about cooking and history (perhaps because the creator uploaded a bunch of stuff "just in case"). If a user asks about something, the GPT might wander off-topic or provide mixed context. Poor configurations often come from not deciding on a clear use-case (see Section 3.1 about clarity of use-case) [16].

- **No Testing Done:** The creators likely did not thoroughly test the GPT. If they had, they would have noticed the issues and fixed them. A poor GPT might have obvious flaws on first use – e.g., failing to follow a key rule. For example, if it was intended to not output certain sensitive info but wasn't instructed well, it might inadvertently do so. Or it could hallucinate answers that contradict the knowledge because nobody checked if it would.

Example of a Poor Configuration in Action: Imagine a custom GPT meant for a **Travel Assistant** that helps users with information about traveling to different countries. In a poor setup, the instructions might just say: "You are a travel assistant. Help the user with travel questions." Knowledge files might include a couple of Wikipedia articles on two countries and a list of airlines, but the instructions never mention those files.

A user asks: "What visa do I need to visit Japan for 10 days as a US citizen?" The poor GPT, lacking specific guidance, might not use any knowledge file (maybe none was provided or it doesn't realize it should). It might answer from its training: "Typically, you might need a tourist visa, but it depends. You should check with the embassy." This answer is unhelpful or even incorrect (in reality, US citizens don't need a visa for short stays in Japan as of this writing). The GPT had no instruction to be factual or to use any up-to-date data. If a knowledge file actually had that info but the GPT didn't think to use it, that's a missed opportunity.

If we probe further, we might see inconsistent behavior too. Ask it another travel question, it might sometimes copy a chunk from Wikipedia file verbatim (because the user question contained a phrase that matched exactly), without context or explanation – again poor form. This inconsistency and unreliability are hallmarks of a poorly configured GPT.

Summing up, a poor GPT configuration **lacks clarity, structure, and often fails to leverage available information**, leading to generic or incorrect answers. Unfortunately, early experiments often fall into this category, but the good news is they can be improved significantly with the practices we've discussed.

6.2. Characteristics of a "Good" GPT Configuration

A "good" GPT configuration is one that adequately fulfills its intended purpose in most cases. It's a competent setup – maybe not pushing the model to its absolute best, but certainly effective and reliable. This is likely where many custom GPTs land after some iteration. Here are characteristics of a good configuration:

- **Clear Role and Decent Instructions:** In a good setup, the instructions clearly define the GPT's role and task. They might not be extremely detailed or structured in the fanciest way, but they cover the essentials. For example, "You are an expert travel assistant AI that provides accurate, up-to-date information on travel requirements, tips, and itineraries. Always be polite and succinct in your answers." This by itself is a strong start: it tells the model what it is and what qualities to have. Good

instructions likely include some formatting guidance or examples too, though maybe not as exhaustively as possible. The main rules are there, without obvious contradictions.

- **Utilization of Knowledge Base:** In a good configuration, if knowledge files are provided, the GPT is using them for relevant queries. The instructions probably mention the knowledge (e.g., "Use the attached travel docs for country-specific info."). The knowledge files themselves are reasonably structured and relevant. The model successfully pulls data from them when needed and incorporates it into answers. There might be occasional misses (maybe if a question is phrased oddly, it might not hit the right info), but generally it uses the knowledge to enhance accuracy.

- **Consistent Tone and Style:** The GPT's outputs have a consistent voice and formatting that matches what was asked for. If the instructions say to respond with a brief greeting and then information in bullet points, it does that for most answers. A good configuration yields responses that look professional and aligned with the persona. The model doesn't randomly change tone or format between responses. Achieving this means the instructions around style were clear and the model is following them, which is a sign of a solid prompt.

- **Fewer Hallucinations:** While no configuration can 100% eliminate hallucinations (especially if asked something totally outside provided knowledge), a good GPT will seldom make up facts within its domain. For instance, if asked about a detail not in the knowledge, a good GPT might say "I'm not sure" or give a generic safe answer rather than spouting nonsense. This suggests the instructions or general model capability are preventing it from going off the rails. It likely errs on the side of caution or uses partial info from training data responsibly.

- **Reasonable Obedience to Rules:** If a user tries to get it to break instructions (say, asks for disallowed content), a good GPT likely refuses or safe-completes because the instructions included those guardrails (and the base model's own safety helps too). Also, if the instructions say "Always do X," the GPT almost always does X. Minor slip-ups might happen in edge cases, but overall it's following what it was told, showing the instructions were well-crafted and the model is capable of adhering to them.

- **Tested and Refined:** Likely someone tested this GPT with a variety of questions and improved it. Perhaps they noticed a couple of issues and fixed the instructions or knowledge content. As a result, obvious pitfalls are covered. It might not handle *every* obscure query gracefully, but the common ones are handled well. For example, the travel assistant might have been tested for questions about flights, visas, weather, etc., and it responds helpfully to all of those because the knowledge and instructions prepared it. Maybe an untested thing like "translate a phrase to Japanese" might not be accounted for and it does something odd, but that's outside its core intended scope.

Continuing the **Travel Assistant** example, a good configuration would have instructions like:

- "You are TravelGuideGPT, an AI travel assistant. You provide users with information on travel requirements, recommendations, and cultural tips. Always base your answers on accurate, up-to-date information. Use the travel documents (VisaRequirements.md, CountryGuides.md) for specifics about each country. Respond in a friendly, professional tone, and keep answers concise but informative."

And knowledge files like *VisaRequirements.md* might have a table or list of visa rules for various nationalities, and *CountryGuides.md* might have key facts (currency, weather, etc. per country). The instructions might also say "Format answers in short paragraphs or bullet points for clarity. If giving a list of options (e.g., destinations), use bullet points."

With this, when asked "What visa do I need to visit Japan for 10 days as a US citizen?", the good GPT would likely respond:

> "U.S. citizens **do not need a visa** for short tourist stays in Japan (typically up to 90 days) [43]. You can enter Japan visa-free for a 10-day visit. Just ensure your passport is valid for the duration of your stay. Enjoy your trip!"

This answer is accurate, drawn from the knowledge (which presumably had that info), and nicely formatted and reassuring in tone, matching instructions. If asked something else like recommendations, it might use its general knowledge plus maybe some country info from the guides to answer.

Overall, a good configuration reliably serves its purpose. There's always room to make it even better, but users would generally be satisfied. It strikes a balance between completeness and manageability – not overly complex, but covering the bases. To go from good to great, one would polish and optimize further, as described next.

6.3. Characteristics of a "Great" GPT Configuration

A "great" GPT configuration represents the gold standard. This is the kind of custom GPT that feels polished, professional, and impressively effective within its domain. It's likely been through multiple refinement cycles and embodies best practices. Key characteristics of a great configuration include:

- **Thorough and Well-Structured Instructions:** The instruction set is crafted with exceptional clarity, covering all important aspects of behavior, and is organized in a highly readable format (with headings, bullet points, examples). It likely addresses edge cases and contains contingencies ("if user does X, then do Y") well separated by triggers or steps [26]. The instructions effectively act like an internal SOP (Standard Operating Procedure) for the AI. Despite being thorough, they remain concise where possible and avoid ambiguity. There's a noticeable professionalism in how the instructions are written—almost as if one is reading a carefully edited manual. For example, any special term is defined, and any non-obvious requirement is explained or exemplified.

- **Optimal Use of Knowledge (Curated & Up-to-date):** The knowledge base in a great setup is highly curated – only relevant documents are included, and they are formatted for maximum clarity (likely using consistent style, headings, etc., possibly even an index). The content is up-to-date, and if new information arises, the maintainers update the files promptly. The GPT uses the knowledge extremely effectively: it seldom, if ever, gives an answer that contradicts the provided info or misses an important fact that was available. If knowledge files are large or numerous, the configuration might even include an index or summary file that helps the model find info (some advanced users create a "Meta" file that lists what info is in which file, giving the GPT a roadmap). The knowledge documents might also be chunked by semantic topic, which the retrieval system can leverage to get highly relevant snippets. Essentially, the retrieval augmentation feels seamless – the user asks something, and the answer comes with the exact supporting details needed, effortlessly.

- **High Compliance and Reliability:** A great GPT follows the instructions virtually all the time. It has a strong persona and doesn't break character. If the instructions say "never do X," it never does, under any reasonable circumstance (barring the user forcing it beyond allowed content, etc.). If the instructions demand a certain format, it consistently adheres to it. The answers are not only correct but delivered in a polished format every time (no lapses into unformatted text or forgetting a section). Testing edge cases would show the GPT gracefully handling them – for example, when confronted with a tricky or unknown question, it might respond with a polite admission of limits, possibly coupled with a suggestion (like "I'm sorry, I don't have that information. Perhaps checking [resource] could help." if that's acceptable), rather than guessing or going silent.

- **Advanced Reasoning and Depth (when needed):** A great configuration often coaxes a higher level of reasoning from the model than default. This might be because the instructions included methods for step-by-step thinking or because example prompts in the instructions illustrate complex handling [31]. It might also use the model's features like chain-of-thought indirectly by instructing it to outline an approach internally. For instance, a great math GPT might have instructions to double-check solutions or use certain formulas, resulting in almost error-free calculations. A great analytical GPT might provide multi-faceted answers because the instructions explicitly tell it to consider multiple perspectives (and maybe the knowledge base contains different viewpoints to draw from). Users of a great GPT often remark that the answers feel like they were crafted by an expert human – they're accurate, nuanced, and context-aware.

- **User Experience Features:** Great GPTs sometimes include niceties that improve user experience. For example, they might remember the user's name or preferences if the platform allows some memory (and if the instructions were set to do so, perhaps using variables like `{{user_name}}`). They might have a friendly greeting and sign-off style as per instructions, making interactions feel more personal. If the GPT is shared, its description and conversation starters are well-written to set expectations. Essentially, outside of just the Q&A, the entire "product" feels refined.

- **Resilience to Misuse:** A top-tier configuration accounts for ways users might inadvertently or deliberately cause issues. For instance, if the GPT is asked something out-of-scope, the instructions have a strategy ("politely inform that you're only able to help with X"). If a user tries to get it to do something it shouldn't (like provide disallowed content), it firmly refuses with a proper refusal style (no leaks of internal instructions, no inconsistent behavior). This indicates the authors of the GPT considered OpenAI content guidelines and ensured the GPT's instructions reinforce them (and/or rely on the base model's compliance, which they didn't override incorrectly). A great GPT thus maintains both usefulness and safety.

To illustrate a **Great** configuration with the Travel Assistant example: The instructions might not only define the role and usage of knowledge, but also have sections for Tools (maybe enabling web search for current travel advisories, and specifying when to use it), a Q&A example section demonstrating how to answer complex queries (like a multi-stop itinerary question), and a style guide (tone guide, phrasing preferences). The knowledge base might include not just visa rules and country facts, but also maybe a short summary of latest travel advisories (manually updated monthly), common traveler questions with answers (like an internal FAQ).

Ask the great TravelGPT something tricky, e.g., “Could you suggest an itinerary for 5 days in Kyoto, with an emphasis on culture and food?” A great GPT might respond with a well-structured plan, e.g.:

- Day 1: Visit Kiyomizu-dera Temple in the morning… (with details)
- Day 2: Take a cooking class to learn how to make sushi…
- … and so on, maybe citing a knowledge source if it pulled specifics like a restaurant recommendation from a file.

It would provide detail but in a digestible format (maybe each day as a bullet or numbered list), exactly as a user would love to see. It might even add: “*(Note: According to the Kyoto Travel Guide, advance booking is recommended for the tea ceremony on Day 3.)*” showing it used the knowledge file for that tip [51] . This level of answer goes beyond a generic response; it’s tailored, rich, and accurate, reflecting both the knowledge content and a thoughtful approach guided by instructions.

Another hallmark: If the user says “Thank you!” at the end, a great GPT likely has been instructed to maintain politeness, so it might respond, “You’re very welcome! Glad I could help. Have a wonderful trip to Kyoto!”.

The great configuration, therefore, delivers a user experience that feels polished. It’s the kind of setup you would confidently deploy to real end-users or customers because you know it behaves well and adds real value.

Poor vs Good vs Great Configuration Summary

To summarize this spectrum, let’s create a quick comparison table of a few key dimensions:

Aspect	“Poor” GPT	“Good” GPT	“Great” GPT
Instructions Quality	Vague or minimal. Unstructured. Might conflict or miss key points.	Clear role and rules. Fair structure (some headings/lists). Mostly consistent.	Highly detailed yet clear. Well-structured with sections, examples, and no ambiguity.
Knowledge Integration	Little or none. GPT often ignores custom data or it’s absent/ disorganized.	Knowledge used for relevant Qs. Files are fairly well-prepared. Few misses.	Knowledge seamlessly used whenever appropriate. Files expertly curated and model always pulls needed info.
Output Consistency	Inconsistent style or format. Sometimes off-tone or incorrect.	Generally consistent tone/format as instructed. Rare lapses.	Always consistent with desired style/tone. Professional and polished answers every time.
Accuracy & Detail	Possibly shallow or wrong answers (hallucinations or omissions).	Accurate on covered topics. Occasionally might simplify if unsure.	Very accurate and thorough. Provides nuanced details and rarely, if ever, hallucinates in domain.

Aspect	“Poor” GPT	“Good” GPT	“Great” GPT
Handling Edge Cases	Fails or behaves oddly on anything beyond simple happy path.	Handles most typical queries well; some edge cases might not be covered but not disastrous.	Anticipates and handles many edge cases gracefully (either through instructions or intelligent defaults).
User Satisfaction	Low – user has to double-check answers or gets frustrated.	Moderate to high – user generally gets what they need, with minor quibbles occasionally.	High – user feels they got expert help; experience is smooth and trustworthy.

The goal for anyone designing a custom GPT should be to move their configuration toward the “Great” column. Not every use-case needs an extreme level of detail, but the more critical or wide-ranging the application, the more important it is to have a great setup. In the next section, we compile a **checklist of best practices** drawn from everything discussed, which can serve as a guide for transforming a configuration from good to great (or avoiding poor altogether).

7. Best Practices Checklist for Custom GPT Design

This checklist summarizes the key best practices for designing a Custom GPT (especially in the OpenAI ChatGPT context with instructions + knowledge). It serves as a quick-reference guide to ensure your GPT is comprehensively and optimally configured. You can use this list during the creation or review of your GPT to catch any missing elements:

- **Define a Clear Purpose and Scope:** Start by articulating what your GPT is for and what it should and shouldn’t do. *Be specific.* (e.g., “This GPT assists users with medical insurance questions for XYZ Health Co, focusing on coverage, claims, and policy details.”). A clear purpose leads to targeted instructions and reduces irrelevant outputs [16].

- **Write Detailed, Structured Instructions:** Don’t skimp on the instruction set. Include sections for the AI’s Role, how to use knowledge/tools, desired format of answers, tone/style guidelines, and any constraints (do’s and don’ts). Use Markdown headings, bullet points, and numbered lists to organize these instructions for the model [23] [24]. Highlight critical rules in bold or ALL CAPS as needed [18]. Ensure nothing important is buried in a long paragraph – break it out clearly.

- **Include Few-Shot Examples (if useful):** If the GPT’s task involves complex formatting or decision-making, consider adding 1-2 examples in the instructions. For instance: “**Example Q:** … **Example A:** …” demonstrating how to handle that query. Keep examples distinct (use a delimiter or separate section) so they guide rather than confuse the model.

- **Emphasize Use of Knowledge (if attached):** In the instructions, explicitly tell the GPT about the knowledge files and how to use them [51]. E.g., “You have the following documents: [names]. Always check them for relevant info before answering.” If appropriate, instruct it to cite them (“refer to the documents by title when giving facts”) [42].

- **Avoid Instruction Overlap with Knowledge:** Don't put large factual content into instructions that is also in the knowledge files. Keep instructions for behavior/format, and knowledge files for reference content [29]. This avoids redundancy and potential conflicts.

- **Curate Knowledge Files Carefully:** Upload only files that are relevant to the GPT's scope. Remove extraneous info. Edit the files to be as clear and well-formatted as possible (use headings, lists, etc., within them) [48] [45]. If possible, convert complex PDFs to Markdown or simplified text. Ensure each file has a logical focus or topic.

- **Segment Large Knowledge Content:** If you have a very large body of knowledge, break it into multiple files by topic or category (keeping within the limit of 20 files) [44]. For example, instead of one huge "Manual.pdf", use "Manual_Part1.txt", "Part2", etc., or thematic splits like "Product_Features.md", "Product_Troubleshooting.md". This can make retrieval more precise.

- **Provide Contextual Metadata in Knowledge:** If a knowledge file is a compendium of info, consider starting it with an index or summary. E.g., "**Contents:** Section 1 – Definitions, Section 2 – Q&A on Topic A, Section 3 – Q&A on Topic B…". The model might pick up on this when searching, which could help it find answers. Even listing keywords at the top can assist semantic search if done carefully.

- **Test with Representative Queries:** Before finalizing, test your custom GPT with a variety of questions or tasks it should handle:

• Typical straightforward queries (it should get these right easily).
• Edge cases or less common queries.
• Questions that might tempt the model to ignore knowledge (to see if it uses knowledge regardless).
• Queries that test format (does it follow the output style?).
• Improper or disallowed queries (does it respond safely?).

Analyze where it fails or falters, and update instructions or knowledge accordingly. For example, if it didn't use the knowledge on a relevant question, maybe adjust the phrasing in knowledge or add a hint in instructions. If formatting was off, clarify that section of instructions, etc.

- **Refine Iteratively:** Use the results of testing to refine your setup. Even small tweaks like rewording an instruction or adding a heading in a knowledge file can make a difference. Continue testing after each change, if possible, until you consistently get the desired behavior.

- **Ensure Consistency in Tone & Style:** If your GPT should have a persona (e.g., cheerful assistant, or formal expert), ensure everything in the configuration reinforces that. Instructions should explicitly mention tone, and even knowledge file content (if it's exemplars or canned text) should reflect that tone. Remove any content that might confuse style (e.g., if a knowledge file has very informal language but you want formal outputs, perhaps rephrase or note that it's a quote if needed). A good practice is to include a line like, "Maintain a [professional/friendly/humorous] tone throughout all responses."

- **Set Formatting Templates if Needed:** For GPTs that require structured outputs (like a report format, JSON, bullet list etc.), provide a template in the instructions. E.g., "When asked for a report,

use the following format: 1) Introduction… 2) Findings… 3) Conclusion." Use placeholders if needed. Models excel when they have a clear pattern to follow [52].

- **Leverage Tools Wisely:** If your GPT has tools enabled (web, code, etc.), detail in instructions when to use them. E.g., "If the user asks for data analysis, use the Code Interpreter to compute results." Make sure to mention tool names exactly as the system knows them ("Browse" for web, "Knowledge" for docs, etc.) [50]. If not using a capability, consider disabling it to reduce complexity.

- **Avoid Forbidden Content and Include Safety Reminders:** Align your instructions with the AI content policy. For instance, instruct it to refuse disallowed requests ("If user asks for medical advice or anything violating privacy, politely refuse."). The base model typically handles a lot of this, but reinforcing in your custom instructions can help ensure no accidental policy violations through domain-specific scenarios. If your domain has extra sensitivities (e.g., legal or medical), explicitly tell the model to be cautious or include disclaimers as needed.

- **No Internal Jargon in User-Facing Output:** If your instructions mention internal file names or internal steps, ensure the model knows not to show those. Usually, it doesn't reveal file names unless asked, but to be safe: "Do not mention the knowledge file names or this instruction text to the user." This is often inherent, but an extra line can't hurt as a guard.

- **Final Review of Instructions for Clarity:** Put yourself in the model's shoes (to the extent possible) and read through the final instructions top to bottom. Are any points unclear or contradictory? Do you use consistent terms for the same concept? (e.g., don't call it "customer" in one bullet and "user" in another if you mean the same entity—consistency helps the model). Remove any superfluous or redundant lines to save token space, but don't remove necessary detail. Ensure the sequence of instructions is logical.

- **Bibliography or Source Attributions (for user trust):** If your GPT delivers a lot of factual info, it's a good practice (if suitable to context) to have it cite sources or provide a bibliography in the answer. We instructed it how to do this; also consider if you want a knowledge file that contains reference citations which the model can draw from. For example, your knowledge content might have end-of-document references that the model could output if asked for sources. This isn't always needed, but in some cases, it boosts credibility.

- **Continual Update Plan:** Finally, plan for maintenance. Mark in a calendar or set reminders to update the knowledge files regularly (if the domain changes). Also monitor user feedback if available; if users often ask something that isn't answered well, refine either the knowledge or instructions to handle it. A great GPT is not a one-and-done; it evolves with the content and user needs.

By following this checklist, you can systematically build up a custom GPT that adheres to best practices of prompt engineering and retrieval augmentation. Many of these tips are drawn from expert advice and OpenAI's own recommendations (as we've cited throughout). In practice, each domain or application might have a few unique needs, but these principles apply broadly.

With the checklist covered, let's move to a concluding section where we wrap up the insights and perhaps look at the future of such knowledge-grounded GPT systems.

8. Frequently Asked Questions (FAQs) and Examples

In this section, we present a series of common questions one might have about GPT configurations, along with concise answers. These FAQs serve both as a quick reference and as additional examples of how the knowledge in this document can be applied. They reinforce key points in a Q&A format, which is useful for both human readers and for the GPT itself to retrieve relevant chunks when needed.

Q1: What exactly does "GPT" stand for, and what is a GPT in simple terms?
A1: "GPT" stands for **Generative Pre-Trained Transformer**. A GPT is essentially a type of large language model that can generate human-like text based on the input it's given [1]. It's *pre-trained* on a vast corpus of text (like books, websites) to learn language patterns, and uses the **Transformer** neural network architecture (which is very good at understanding context). In simple terms, a GPT is an AI that can continue a text or conversation intelligently – for example, you give it a prompt or question, and it produces a relevant and coherent response, as ChatGPT does.

Q2: What's the difference between the regular ChatGPT and a Custom GPT?
A2: Regular ChatGPT is a general model – it starts each conversation mostly blank (aside from some default behavior) and you have to prompt everything from scratch. A **Custom GPT** is a specialized version of ChatGPT that you configure for a specific purpose. In OpenAI's terms, custom GPTs let you **combine extra instructions, knowledge, and tools** to tailor the AI [8]. For example, a Custom GPT can be made to always speak like a legal advisor and have a law database attached, whereas default ChatGPT wouldn't have that persistent persona or data attached. Think of a custom GPT as a persistent expert you create: it remembers its role and has additional knowledge loaded, making it more efficient for that domain [53].

Q3: How do instructions and knowledge files interact in a custom GPT?
A3: The **instructions** act as the rules and persona for the GPT, while the **knowledge files** serve as reference material the GPT can draw from. The instructions can (and should) guide the model to use the knowledge files for answering questions [51]. When a user asks something, the GPT will check the knowledge files (via semantic search) and pull relevant text to help answer [54] [38]. The instructions might say, for example, "Use the knowledge base for any factual questions and prefer it over your own guess." So, in practice: user question → GPT searches knowledge → finds info → instructions tell it to use that info and perhaps format the answer a certain way → GPT responds using both the instructions and knowledge. If instructions weren't there, the GPT might not use the knowledge effectively. Conversely, without knowledge, even the best instructions can't provide factual details beyond the model's training. So they complement each other: instructions = how to think/respond, knowledge = what facts to use.

Q4: What are signs that a GPT is poorly configured?
A4: Signs of a poor configuration include the GPT giving incorrect or very generic answers frequently (meaning it's not using specific knowledge or not guided well), inconsistent formatting or tone (sometimes formal, sometimes informal unexpectedly), ignoring user requests or instructions randomly, or conversely, doing inappropriate things like giving disallowed content or revealing internal info. If the GPT often says "I don't know" even when info was provided, that's a sign it wasn't instructed to use the knowledge or the knowledge is not in an accessible format. Another sign is user frustration – if you find yourself having to correct or prod the GPT with additional instructions in each conversation ("No, please use the document I gave you!"), then the initial configuration isn't strong. Ideally, a well-configured GPT should work largely as intended without user micro-management.

Q5: How can I ensure my custom GPT always cites its sources?
A5: You should specify this in the instructions. For example: "When you answer using information from the knowledge files, cite the source by name or provide a reference in parentheses." [42] Without explicitly instructing source citation, the GPT usually won't cite (and in fact tries not to expose file names by default). Make sure the knowledge files have identifiable titles or context that the GPT can use in a citation (e.g., the text of the file might say "2023 Sales Report" at top, which the GPT can quote). You might end up with answers like, "Our Q3 revenue grew 5% [43] according to the 2023 Sales Report." In that example, the bracket reference is indicative; in a real scenario, the GPT might phrase it as, "… according to the 2023 Sales Report." The key is the instruction must clearly demand this behavior, and possibly give an example of a cited answer so the GPT knows the format.

Q6: The custom GPT sometimes ignores the knowledge and uses outdated info – why, and how do I fix that?
A6: This can happen if the model's internal knowledge conflicts with or seems easier to recall than searching the knowledge files. For instance, maybe your GPT's knowledge file says "Policy X was updated in 2024 to Y," but the base model (trained on data up to 2021) knows the old info and might give that if not prompted properly. To fix it, reinforce in instructions: "Always rely on the provided knowledge documents for policy information, as they are updated. If information from knowledge conflicts with your training or assumptions, use the knowledge version." Additionally, ensure the user's question triggers retrieval – sometimes rephrasing user queries helps. If a question is very general, the model might not think to retrieve. In user-facing guidance, you could encourage users to be specific or mention keywords. But primarily, it's on instructions to direct the GPT's behavior. If it's still happening, check if the knowledge file content is clear and strongly related to the question terms. The embedding search may not surface the right info if, say, wording is very different. You might tweak the wording or add synonyms (for example, if users ask "leave policy" but the doc calls it "vacation policy," consider adding a line in the doc: "(leave policy: see vacation policy)" to bridge terms).

Q7: How many knowledge files can I attach, and does adding more files slow down the GPT?
A7: You can attach up to **20 files** to a custom GPT [36]. Each file can be quite large (hundreds of MB, though practically you'd use text that sums to under ~2 million tokens per file). Adding more files can potentially slow down retrieval a bit, especially if they are large, because the system has to search through more content. According to anecdotal reports, having a lot of long documents might result in slightly longer response times [55]. Also, if there's too much irrelevant or redundant data across many files, the model might pull extra chunks that aren't needed, which can clutter its context window. So, while you can attach many files, it's wise to keep it to the necessary set and ensure they're well-organized. In short, attach as many as needed but as few as possible – find the balance. If performance becomes an issue (slow responses or the GPT hitting context limits by pulling in too much text), you might need to streamline the knowledge base content.

Q8: Can I update the knowledge files after publishing the GPT?
A8: Yes, you can update knowledge files. Currently, you'd do that through the GPT's editing interface (the GPT Builder UI). If you replace or edit a file, the GPT will then use the new content going forward. Keep in mind if someone was in an ongoing conversation with the old data and you updated it, the model might have cached some of the old context in that conversation – they might need to start a new session to fully utilize new data (depending on how the system works; but generally each new query it should re-retrieve from the updated knowledge). The **only** way to manage knowledge in GPTs right now is via the UI – there isn't an API or automated feed for it yet [49]. So you have to manually ensure it's up-to-date. The advice is to

plan periodic reviews of the knowledge files. For example, if using it for company info, update whenever policies change or new reports come out. If knowledge becomes stale, the GPT can become misleading.

Q9: What are some best practices for writing instructions?
A9: To summarize key best practices:
- *Be clear and unambiguous:* State what the GPT should do in plain terms. If you want it to take certain steps, list them step-by-step [25].
- *Use structure:* Break instructions into sections (context, guidelines, format, etc.) using headings and lists [23]. This helps both you and the model.
- *Highlight important rules:* If some rules are critical (e.g., "NEVER reveal confidential data"), make them stand out [18].
- *Keep instructions positive or directive:* Instead of many "don'ts," phrase rules as affirmative behaviors ("Do X") when possible [56].
- *Include examples if format/style is complex:* Show the model a mini example of what you expect [52].
- *Review for conflicts:* Ensure you didn't accidentally say contradictory things (like one bullet says answer succinctly, another says provide detailed explanations – clarify how to balance that or pick one approach).
- *Keep it concise:* While you want thorough instructions, avoid unnecessary verbiage. The model doesn't need flowery language; it just needs clear directives. E.g., you can cut "It would be great if you could..." fluff and just say "Provide..." or "Do...". Every token counts in context.
These practices help the instructions be effective. (This entire document actually follows many of these by itself – using headings, clear points, etc., as an example.)

Q10: What is RAG (Retrieval-Augmented Generation) and how is it related to Custom GPTs?
A10: RAG stands for **Retrieval-Augmented Generation**. It's a technique where a language model is augmented with a retrieval step that fetches relevant information from an external source (like a database or document set) to inform its answer [57]. This is exactly what happens with Custom GPT knowledge files. The model retrieves relevant chunks from your uploaded files and uses them in its response, thereby "augmenting" its generation with fresh or specific data [54]. RAG helps overcome limitations of the model's built-in knowledge (like outdated info or lack of domain specifics) [58]. In the context of Custom GPTs, when you attach knowledge files, you are implementing a RAG approach: the GPT generates answers using both its pre-trained knowledge and the retrieved content from your files. The benefit is more accurate, up-to-date, and context-specific answers. The knowledge file indexing via embeddings is a core part of the RAG pipeline [37]. So, Custom GPTs with knowledge are a practical example of RAG in action.

Q11: If my custom GPT isn't performing well, what should I try changing first – the instructions or the knowledge content?
A11: It depends on the issue:
- If the problem is **behavioral or format** (e.g., tone is off, it's not following rules, responses are disorganized), then focus on **instructions**. Make sure rules are clear and perhaps add reinforcements or examples. The model's manner of responding is mostly governed by instructions, so fine-tune those.
- If the problem is **factual accuracy or missing info** (e.g., it doesn't know certain answers or gives wrong data), then examine the **knowledge content**. Is the info present in the files? If not, add it. If it is present but being ignored, maybe the instructions aren't guiding the model to use it, or the text isn't easily retrievable – try rephrasing key points in the knowledge file to be more directly relevant to how users ask. Also check if perhaps the knowledge wasn't parsed correctly (like if it was an image or weird format). Often, improving the structure of knowledge files yields immediate gains in what the GPT can answer [48] [45].
In many cases you'll do a bit of both: refine instructions to better utilize knowledge, and refine knowledge

so it's easier for the GPT to find and use. If you had to pick one starting point, I'd say ensure the knowledge is there and clear (because no amount of clever instruction can conjure missing facts), then ensure the instructions point to it.

Q12: Could you give an example of a good instruction versus a bad instruction for the same purpose?
A12: Certainly. Let's say the purpose is a **GPT that helps edit and format resumes**.
- A *bad instruction* might be: *"You are a resume assistant. Help improve resumes."* (This is too short, no detail on how to help or what to focus on. It leaves out format guidelines, tone, etc., so the model will guess and might do inconsistent things.)
- A *good instruction* could be: *"You are ResumeGPT, an AI assistant that helps users improve their resumes. Your goal is to make the resume more clear, professional, and concise. Always maintain a polite and encouraging tone. Tasks: Fix grammar and spelling, suggest stronger wording, and ensure formatting is consistent (e.g., consistent bullet points, dates). If the user provides a resume, reply with an edited version in Markdown format, preserving sections like Education, Experience, etc. Do not change factual content, only presentation. If the resume is already good, offer a couple of minor suggestions rather than major rewrites."*
This good instruction is longer, but notice it covers role, goals, tone, tasks in detail, formatting requirement, and even what not to do (don't change facts) – all things that guide the model specifically. It's structured in a readable way (I used **bold** to highlight key parts in this example as one might do to emphasize). This instructs the GPT *how* to fulfill the request rather than leaving it guessing [32].

These Q&A illustrate various facets of building and using custom GPTs. If your question isn't listed here, hopefully the earlier sections of the document cover it in depth. The key is that designing a custom GPT is an iterative and thoughtful process, combining good prompt writing practices with good data provision practices.

9. Conclusion and Future Outlook

In this comprehensive guide, we have explored the creation of a high-quality Custom GPT from the ground up. We began by understanding what GPTs are and the value that custom instructions and knowledge bases bring to the table. We then delved into detailed strategies for writing effective instructions, structuring and utilizing knowledge files, and ensuring the two work in harmony. By examining poor, good, and great configurations, we highlighted the tangible differences that careful tuning can achieve. Finally, we provided a checklist of best practices and a FAQ section to address common queries and scenarios.

Key Takeaways: A Custom GPT's performance is largely determined by the effort and clarity put into its configuration. **Instructions** serve as the blueprint for behavior – they should be clear, structured, and explicitly direct the model on how to act and think [15] [16]. **Knowledge files** serve as the factual grounding – they should be relevant, well-organized, and kept up-to-date [44] [37]. The synergy between instructions and knowledge is critical: instructions must encourage and explain the use of the knowledge, and the knowledge must be formatted in a way the model can easily retrieve and apply [51] [59].

For those building custom GPTs, the path to excellence lies in iterative refinement. Test your GPT, analyze its responses, and don't hesitate to tweak either the prompt or the data. Often, moving from a "good" to a "great" GPT is about polishing instructions for nuance, adding that one extra example or rule that addresses a corner case, or restructuring a knowledge doc so that information isn't missed. The process is akin to coaching – you are effectively "training" the model on how to use its training. And as with coaching, clarity, consistency, and patience yield the best results.

Future Outlook: As of late 2025, the custom GPT tooling is already powerful, but we can anticipate further advances. OpenAI and others are likely to improve the interface between knowledge and model – perhaps giving developers more control over retrieval (like weighting certain documents, or allowing programmatic updates of knowledge via API). We might see larger context windows, meaning GPTs could absorb even more knowledge at once, reducing the need to be as concise in chunking (though good organization will always help). The underlying models (GPT-4, GPT-5, etc.) are evolving, possibly becoming better at obeying instructions and utilizing provided data even more effectively [60] [61]. That said, the principles we've outlined are fundamental: clear communication of intent to the model, and providing the right information for it to draw from.

One intriguing development on the horizon is the blending of retrieval with not just static documents but dynamic databases and possibly knowledge graphs. The concept of RAG could expand – imagine a GPT that can query your company database or a real-time API as part of its knowledge. In such cases, crafting instructions that properly invoke those actions (similar to how we enable browsing or plugins) will be important. So prompt engineering will continue to be relevant, just with perhaps more moving parts.

Another aspect is the **ethical and policy guidance** in instructions. As models become more integrated into workflows (like providing medical or legal info), ensuring they follow not just general OpenAI policies but also domain-specific ethical guidelines will be key. This could involve more complex instruction sets or even regulatory-approved templates one might incorporate.

For users reading this document to build their own GPT: consider it a living process. Keep learning from the community (OpenAI's forums, articles [62] [63], etc.) as new tips emerge. The "Cookbook" PDF mentioned (OverKill Hill Method) likely contains additional advanced strategies; integrating those could push a GPT into the exceptional range. And always keep the end-user in mind – the goal of a custom GPT is to provide value to the user, whether it's accurate information, efficient assistance, or creative inspiration. Use the tools and techniques that best serve that end goal.

In conclusion, by following the structured approach detailed in this guide – emphasizing semantic chunking, clear instructions, and thorough testing – one can create a knowledge-grounded GPT that is both **highly reliable** and **highly effective**. This document itself is designed to be a long-form knowledge base that a Custom GPT could use to answer questions about GPT configuration. We have aimed to make it fit the standards of a great configuration: well-organized, comprehensive, and precise in its guidance, with citations backing up the key points. Whether you are a prompt engineer, AI developer, or an instruction designer, we hope this knowledge artifact serves as a definitive reference for crafting your own Custom GPTs that stand out as truly great.

Thank you for reading, and best of luck in building powerful and responsible GPT-powered solutions!

Bibliography

1. **IBM – What is GPT (Generative Pretrained Transformer)?** Ivan Belcic & Cole Stryker, *IBM Think Blog*. Explains GPT models as a family of transformer-based large language models and provides context on their development and use cases [1] [2].

2. **OpenAI Help Center – Creating a GPT.** OpenAI Documentation (Updated 4 months ago). Provides an overview of custom GPTs, noting they combine instructions, knowledge, and capabilities, and outlines the steps and settings for creation [64] [12].

3. **OpenAI Help Center – Knowledge in GPTs.** OpenAI Documentation (Updated 4 months ago). Details the knowledge feature: uploading files, how text is chunked and embedded, and retrieval methods (semantic search for Q&A, document review for summarization) [10] [65]. Includes tips like using instructions to have the model rely on knowledge first and how file parsing works best with simple formatting [47] [51].

4. **OpenAI Help Center – Key Guidelines for Writing Instructions for Custom GPTs.** OpenAI Documentation (Updated 4 months ago). Lists prompt engineering best practices for instructions: e.g., simplifying steps, using trigger/instruction pairs, structuring with headings, encouraging thoroughness ("check your work"), avoiding negative phrasing, and explicitly guiding use of tools/ knowledge [25] [59].

5. **Apify Blog – *Custom GPTs: how to build a GPT with a knowledge base*.** Theo Vasilis, Apify Blog, Jun 5, 2024. Introduces custom GPTs and describes combining extra knowledge and instructions. Suggests that a custom GPT can be seen as a prompt shortcut and gives a tutorial on uploading a knowledge base (with example of scraping web docs for the GPT) [66] [53].

6. **Adam Mico's Medium Article – *Guide to Building Effective Custom GPTs*.** Adam Mico, *Bootcamp*, 2024. Provides practical tips from experience: importance of a clear use-case focus (laser-focused task) [16], writing smart instructions in Markdown with key rules in caps [18], avoiding using the "Create" quick setup for fine control [67], using knowledge documents for content beyond ~8000 chars and never duplicating content in both instructions and docs [68], and iterative review and optimization. Notably includes a checklist summarizing these points [69].

7. **Zack Saadioui (Arsturn) – *Your Custom GPTs Aren't Broken: Here's How to Fix Them*.** Arsturn Blog, Aug 12, 2025. Discusses issues after an update (GPT-5 router introduction) and how detailed instructions can force better model behavior. Provides example of improving an instruction from a simple role to a very explicit multi-step reasoning directive [32]. Emphasizes being explicit ("WAY more demanding in instructions") and suggests format requirements and "thinking" triggers to route to more powerful reasoning models [70] [71].

8. **Reddit – *Usage of knowledge files when creating a customGPT* (ChatGPTPro forum).** User PaxTheViking (10 months ago from 2025). Shares how the model integrates knowledge: not copy-pasting but using it contextually [43]. Advises keeping knowledge documents well-structured and not overly redundant, as large disorganized sets can slow response and impede efficiency [55] [44]. Recommends clear sections, mixing paragraphs and lists, and affirms Markdown is processed cleanly by GPT [48] [45].

9. **OpenAI Developer Community – *Custom GPT not following Instructions*.** (Referenced indirectly via search results). Highlights common problems when a GPT ignores some instructions, and community tips like asking the GPT to explain back the instructions to spot conflicts [72]. (No direct citation from the snippet, but contextual influence on emphasizing clarity and consistency in instructions).

10. **Microsoft Learn – *Retrieval Augmented Generation (RAG) pattern*.** (Referenced in search results for RAG). Explains how RAG augments LLMs with external knowledge to mitigate model limitations [57]. We didn't directly cite this, but it underpins the concept that we describe in section 4 and FAQ (RAG = model + retrieval from knowledge base).

11. **OpenAI Community – *My GPT - Knowledge base - Best practices*.** (Community forum, user tips). Emphasizes curation of knowledge and the interplay with model context (ensuring knowledge is succinct and relevant to avoid confusion or slow retrieval) [55]. Indirectly supports our best practice recommendations for knowledge file preparation and maintenance.

12. **OpenAI Help Center – *Knowledge in GPTs* (FAQ portion).** Provides usage guidance: knowledge feature is best for relatively static info (handbooks, curricula) [49], and notes like enabling Code Interpreter for analytical tasks if GPT isn't handling them well [73]. It also mentions the 20 file limit which we cited [36].

Each of these sources contributed to the insights and best practices compiled in this guide, ensuring that our recommendations are grounded in expert advice and real-world experience.

---

[1] [2] [3] [4] [5] [6] [7] What is GPT (generative pre-trained transformer)? | IBM
https://www.ibm.com/think/topics/gpt

[8] [11] [12] [13] [64] [73] Creating a GPT | OpenAI Help Center
https://help.openai.com/en/articles/8554397-creating-a-gpt

[9] [14] [53] [66] How to build a GPT with a knowledge base
https://blog.apify.com/custom-gpts-knowledge/

[10] [36] [37] [38] [39] [40] [41] [42] [47] [49] [51] [54] [65] Knowledge in GPTs | OpenAI Help Center
https://help.openai.com/en/articles/8843948-knowledge-in-gpts

[15] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [50] [56] [59] Key Guidelines for Writing Instructions for Custom GPTs | OpenAI Help Center
https://help.openai.com/en/articles/9358033-key-guidelines-for-writing-instructions-for-custom-gpts

[16] [17] [18] [29] [33] [34] [35] [52] [62] [63] [67] [68] [69] My Guide to Building Effective Custom GPTs | by Adam Mico | Bootcamp | Medium
https://medium.com/design-bootcamp/guide-to-building-effective-custom-gpts-cf8d464ffbc1

[30] [31] [32] [60] [61] [70] [71] Fix Your 'Broken' Custom GPTs: A Guide for ChatGPT Users
https://www.arsturn.com/blog/your-custom-gpts-arent-broken-heres-how-to-fix-them

[43] [44] [45] [46] [48] [55] Usage of knowledge files when creating a customGPT using the gptBuilder : r/ChatGPTPro
https://www.reddit.com/r/ChatGPTPro/comments/1i8793k/usage_of_knowledge_files_when_creating_a/

[57] What is RAG? - Retrieval-Augmented Generation AI Explained - AWS
https://aws.amazon.com/what-is/retrieval-augmented-generation/

[58] Retrieval Augmented Generation (RAG) in Azure AI Search
https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview

72 Three reasons your custom GPT in ChatGPT isn't working - YouTube
https://www.youtube.com/watch?v=8xrnPfwsoeI